

...

In this task you are to write a mini python library(a python file with defined functions like calculator you wrote in the previous class)

You are a Hockey coach of an olympic team. There are 10 players in your team. You need a manager who will help you manage your team better.

1. Define a dictionary(in ipynb notebook) of your team. The key must be a name and the value with four fields(keys):-

```
height - in metres
position (Left Back/Right Back/Middle Back/Goalkeeper/Front Left/Front Right/Front Center)
physique - (explosive, stamina, all-round)
nature - (realist/idealist)
age - in years
```

Example of two players dictionary is below (you must draft a team of your choice with 10 players):

```
{ "sai": {"height": 1.6, "position": "Front Left", "physique": "explosive", "nature": "idealist" },
  "aditya": {"height": 1.5, "position": "Front Center", "physique": "all-round", "nature": "realist" } }
```

2. Write a separate python file "hockeyTeamManager.py". In this you must write the following functions:

- Get the average height and age of a team.
- Get number of explosive/stamina/all-rounders in a team. (print in any format you desire)
- Find the position that two players play (if any)
- Find the position that no player plays (if any)
- Find the nature of the team. Based on number of realists and idealists if 70% of the team are idealists then team is idealist. (same for realist)

Example:

```
if 7/10 are idealists then team is idealist. (same for realist)
if 6/10 or 5/10 are idealist or realists then team is a "mix"
```

- Find the overall physique of the team. The physique that occurs most number of times is the overall physique.
- If a new player is to be added into the team then find the maximum age allowed so that it does not exceed 24 years.
- Write a print function to print in a clear way all the players with their respective positions. (example: <Name> Plays in position <position>)
- Write a function to find all those athletes who are of a particular physique (take physique as argument)
- Write a function to find all those athletes who are of a particular nature (take nature as argument)

Example for above two (i,j):

```
>hockeyTeamManager.GetPlayersWithPhysique(playerDictionary, physiqueToFilter)
Should print name of players with physique are given in argument (explosive/stamina/all-round)
```

3. Now import this python file into the notebook and use all these functions one after the other. Note: these functions must not be seen anywhere in the notebook. They should be written in a separate file.

...

```
#question 1
```

```
team={
```

```
"rajesh":{"height":5.9, "position":"Front Left", "physique": "explosive", "nature":"idealis  
"pavan":{"height":5.4, "position":"left back", "physique": "all-round", "nature":"realist",  
"vardhan":{"height":5.2, "position":"right back", "physique": "explosive", "nature":"ideali  
"kumar":{"height":1.5, "position":"right front", "physique": "all-round", "nature":"realist  
"siva":{"height":4.9, "position":"goal keeper", "physique": "explosive", "nature":"idealist  
"vijay":{"height":4.5, "position":"Front Center", "physique": "all-round", "nature":"realis  
"surya":{"height":5.5, "position":"Front Left", "physique": "explosive", "nature":"idealist  
"praveen":{"height":6.0, "position":"Front Center", "physique": "all-round", "nature":"real  
"nandhu":{"height":5.6, "position":"Front Left", "physique": "explosive", "nature":"idealis  
"ganesh":{"height":5.5, "position":"Front Center", "physique": "all-round", "nature":"ideal  
}
```

```
#question 2(a.1)
#Get the average height of a team.
def avg_height(a):
    he=0
    length=len(a)
    for name,i in a.items():
        he=he+i['height']
    avhe=he/length
    return(avhe)
print(avg_height(team))
```

...

Explanation:

Function Definition:

`def avg_height(a):` defines a function named `avg_height` that takes one argument, `a`. This function performs the following steps:

`he=0:` Initializes a variable `he` to store the total height of the team members.

`length=len(a):` Calculates the number of team members (length of the dictionary `a`) and stores it in the variable `length`.

`Iterating Through Team Members:`

`for name,i in a.items()::` Iterates through the dictionary `a` which represents the team.

`name:` Represents the name of a team member.

`i:` Represents the information about the team member, which is likely another dictionary containing the member's details.

`Calculating Total Height:`

`he=he+i['height']:` Adds the height of the current team member (`i['height']`) to the total height (`he`).

`avhe=he/length:` Calculates the average height by dividing the total height (`he`) by the number of team members (`length`).

`return(avhe):` Returns the calculated average height.

Function Call:

`print(avg_height(team)):` Calls the `avg_height` function with the `team` dictionary as input and prints the result.

Assumptions:
The team dictionary contains information about team members, with each member represented by an inner dictionary. The height of each team member is stored in the 'height' key of the inner dictionary.

Potential Issues:

Typo: There's a typo in the code: 'height' should be 'height'.

Data Type: Ensure that the heights are stored as numerical values (integers or floats) for the calculation to work correctly.

...

5.0

```
#question 2(a.2)
#Get the average height of a team.
def avg_age(a):
    ag=0
    length=len(a)
    for name,i in a.items():
        ag=ag+i['age']
    avag=ag/length
    return(avag)
print(avg_age(team))
```

```
...
```

Function Definition:

def avg_age(a): defines a function named avg_age that takes a single argument a.
Variable Initialization:

ag=0: Initializes a variable ag to store the total age of all team members.
length=len(a): Calculates the number of team members (length of the dictionary a).
Iterating Through Team Members:

for name,i in a.items(): Iterates through the dictionary a, where:
name is the name of the team member (key).
i is the information about the team member (value), which is likely another dictionary cont
Calculating Total Age:

ag=ag+i['age']: Adds the age of the current team member (i['age']) to the total age (ag).
Calculating Average Age:

avag=ag/length: Calculates the average age by dividing the total age (ag) by the number of
Returning Average Age:

return(avag): Returns the calculated average age.
Printing Average Age:

print(avg_age(team)): Calls the avg_age function with the team dictionary and prints the re
Explanation

The code calculates the average age of a team. It assumes that the team is a dictionary whe

The function avg_age takes a dictionary a representing the team as input.
It initializes variables ag to store the total age and length to store the number of team r
It iterates through the team members, adding each member's age to the total age.
It calculates the average age by dividing the total age by the number of team members.
It returns the calculated average age.
The code calls the avg_age function with the team dictionary and prints the result.
In essence, the code efficiently computes the average age of a team given a dictionary cont
...

⇒ 24.0

```
#question 2(b)
# Get number of explosive/stamina/all-rounders you a team.(print in any format you desire)
```

```
def phv count(a):
```

```

exp=0
sta=0
all=0
for name,i in a.items():
    if i['physique']=="explosive":
        exp=exp+1
    elif i['physique']=='stamina':
        sta+=1
    else:
        all+=1
    return(exp,sta,all)
print(phy_count(team))
print("explosive=",phy_count(team)[0])
print("stamina=",phy_count(team)[1])
print("all-round=",phy_count(team)[2])

...

```

Understanding the Code:

Purpose:

To count the number of athletes in a team with different physiques: explosive, stamina, and all-round.

Code Breakdown:

Function Definition:

`def phy_count(a):` defines a function named `phy_count` that takes a dictionary `a` as input, pre-initializing counters.

`exp = 0, sta = 0, and all = 0` initialize counters for explosive, stamina, and all-round athletes.

Iterating Through Athletes:

`for name, i in a.items():` iterates through the dictionary `a`, where:
`name` is the athlete's name.
`i` is the athlete's information, likely containing a 'physique' key.

Counting Athletes:

`if i['physique'] == "explosive":` checks if the athlete has an explosive physique and increments `exp`.
`elif i['physique'] == 'stamina':` checks if the athlete has a stamina physique and increments `sta`.
`else:` if the physique is neither explosive nor stamina, it's assumed to be all-round, and `all` is incremented.

Returning Counts:

`return (exp, sta, all)` returns a tuple containing the counts of explosive, stamina, and all-round athletes.

Printing Results:

`print(phy_count(team))` calls the `phy_count` function with the `team` dictionary and prints the results.
`print("explosive=", phy_count(team)[0]), print("stamina=", phy_count(team)[1]), and print("all-round=", phy_count(team)[2])` print the counts for each physique.

Output:

The output will be something like:

```

(2, 3, 1)
explosive = 2
stamina = 3
all-round = 1

```

This indicates that there are 2 explosive athletes, 3 stamina athletes, and 1 all-round athlete.

Note:

The code assumes that the team dictionary is defined elsewhere and has the same structure as
The code doesn't handle cases where the 'physique' key is missing or has an unexpected value
The format of the output can be customized as needed.
...

```
⇒ (1, 0, 0)  
   explosive= 1  
   stamina= 0  
   all-round= 0
```

```
#question 2(c)
#Find the position that two players play(if any)
pos=[]
for name,i in team.items():
    pos.append(i['position'])
print(pos)

...
```

Understanding the Code

The provided code snippet is designed to extract the positions of all players from a given

Breakdown:

Initialization:

`pos = []` creates an empty list named `pos` which will be used to store the positions of the p
Iterating Through the Team:

`for name, i in team.items():` iterates through the team dictionary.
`name` represents the player's name (key).
`i` represents the player's information (value), which is likely another dictionary containin
Extracting Positions:

`pos.append(i['position'])` appends the player's position (accessed by `i['position']`) to the
Printing Positions:

`print(pos)` prints the final list of player positions.
Purpose:

While the code extracts the positions of all players, it doesn't directly find the position

Example:

If the team dictionary looks like this:

```
Python
team = {
    "Player1": {"position": "forward"},
    "Player2": {"position": "defender"},
    "Player3": {"midfielder"}
}
```

Use code with caution.

The output of the code would be:

```
['forward', 'defender', 'midfielder']
```

To find the positions of two specific players, you would need to:

Identify the two players (e.g., by their names).
Access their position information from the team dictionary.
Compare the positions of the two players.
Additional Considerations:

The code assumes that the team dictionary is defined somewhere in the code before this snip
The structure of the player information within the dictionary is assumed to be a dictionary

The code doesn't handle cases where a player's position is missing or if there are errors i
...

↩ ['Front Left', 'left back', 'right back', 'right front', 'goal keeper', 'Front Center',
...




```
#question 2(d)
#Find the position that no player plays(if any)
for i in pos:
    if pos.count(i)==1:
        print(i)
        break
    else:
        print("no player plays")
```

...

Understanding the Code:

The code aims to find a position that no player plays in a given list of positions.

Breakdown:

for i in pos:: This line iterates through each position in the list pos.
 if pos.count(i)==1:: This checks if the current position i appears only once in the list pos.
 print(i): Prints the position as it's unique.
 break: Stops the loop as the first unique position is found.
 else:: If the current position i appears more than once in the list:
 print("no player plays"): Prints a message indicating no unique position found.
 Explanation:

The code essentially counts the occurrences of each position in the list. If a position appears only once, it is printed.

Limitations:

This code assumes that there's at most one unique position. If multiple unique positions exist, it only prints the first one. It doesn't handle empty lists or lists with only one element correctly.

Improved Version:

Python

```
def find_unoccupied_position(pos):
    unique_positions = set(pos)
    if len(unique_positions) == len(pos):
        print("All positions are occupied.")
    else:
        for p in unique_positions:
            if pos.count(p) == 1:
                print("Unoccupied position:", p)
```

Use code with caution.

This improved version:

Handles cases where all positions are occupied.
 Finds all unique unoccupied positions.
 Uses a set to efficiently find unique positions.
 Example:

Python

```
pos = ["forward", "defender", "midfielder", "defender"]
find_unoccupied_position(pos) # Output: Unoccupied position: forward
```

Use code with caution.

This would correctly identify "forward" as the unoccupied position.
...

⇒ no player plays
left back

```

#question 2(e)
#Find the nature of the team. Based on number of realists and idealist if 70% of the team i
''' Example
    if 7/10 are idealists then team is idealist.(same for realist)
    if 6/10 or 5/10 are idealist or realists then team is a "mix" '''
def nature(a):
    real=0
    ide=0
    for name,i in a.items():
        if i['nature']=="realist":
            real=real+1
        else:
            ide=ide+1
    return(real,ide)

real, ide = nature(team)
print(nature(team))
if real>ide:
    print("realist")
elif real<ide:
    print("idealist")
else:
    print("mix")

'''

```

Understanding the Code:

Purpose:

Determines the overall nature of a team based on the number of realists and idealists. Classifies the team as "realist", "idealist", or "mix".

Breakdown:

Function nature(a):

Takes a dictionary a as input, representing the team.
 Initializes real and ide to 0 to count realists and idealists.
 Iterates through the team members:
 If a member's nature is "realist", increments real.
 Otherwise, increments ide.
 Returns a tuple (real, ide) containing the counts.
 Main Logic:

Calls the nature function to get the counts of realists and idealists.
 Compares the counts:
 If real is greater than ide, the team is "realist".
 If ide is greater than real, the team is "idealist".
 Otherwise, the team is a "mix".
 Explanation of the Logic:

The code assumes that each team member has a nature attribute which is either "realist" or "idealist". The nature function counts the occurrences of each nature. The main logic categorizes the team based on the majority or equality of realists and idealists.
 Example:

If the team dictionary looks like this:

Python

```
team = {  
    "Alice": {"nature": "realist"},  
    "Bob": {"nature": "idealist"},  
    "Charlie": {"realist"},  
    "David": {"idealist"},  
    "Eve": {"realist"}  
}
```

Use code with caution.

The output would be:

(3, 2)

realist

Because there are more realists (3) than idealists (2), the team is classified as "realist"

Note:

The code doesn't explicitly check for the 70% threshold as mentioned in the comment. It sim
The code could be improved by adding error handling for cases where the nature attribute is
The code could also be made more flexible by allowing different thresholds for classificati
'''

↩ (4, 6)
idealist

#question 2(f)

Find the overall physique of the team. The physique that occurs most number of times is t

```
def overall_physique(a):
    exp=0
    sta=0
    all=0
    for name,i in a.items():
        if i['physique']=="explosive":
            exp=exp+1
        elif i['physique']=='stamina':
            sta+=1
        else:
            all+=1
    if exp>sta and exp>all:
        return("explosive")
    elif sta>exp and sta>all:
        return("stamina")
    else:
        return("all-round")
print("overall physique of the team is:",overall_physique(team))
```

...

Understanding the Code:

Purpose:

To determine the overall physique of a team based on the most frequent physique among its m
Function Breakdown:

Python

```
def overall_physique(a):
```

Use code with caution.

Defines a function named overall_physique that takes a dictionary a as input, presumably cc
Python

```
exp = 0
sta = 0
all = 0
```

Use code with caution.

Initializes three counters: exp for explosive, sta for stamina, and all for all-round physi
Python

```
for name, i in a.items():
    if i['physique'] == "explosive":
        exp += 1
    elif i['physique'] == 'stamina':
        sta += 1
    else:
        all += 1
```

Use code with caution.

Iterates through the dictionary a, incrementing the corresponding counter based on the athl
Python

```
if exp > sta and exp > all:
    return("explosive")
```

```
elif sta > exp and sta > all:
    return("stamina")
else:
    return("all-round")
Use code with caution.
```

Compares the counts of each physique and returns the one with the highest count. If there's Python

```
print("overall physique of the team is:", overall_physique(team))
Use code with caution.
```

Calls the overall_physique function with the team dictionary and prints the result.
Assumptions:

The team dictionary contains athlete information with a 'physique' key, having values 'expl
There are no other possible physique values in the data.
Explanation:

The function calculates the count of each physique type in the team.
It compares the counts to determine the most frequent physique.
It returns the most frequent physique as the overall physique of the team.
Example:

If the team dictionary contains 5 athletes with 'explosive' physique and 3 with 'stamina',

Potential Improvements:

Handle cases where there might be more than three physique types.
Consider using a dictionary to store counts instead of multiple variables.
Implement error handling for cases where the 'physique' key is missing or has unexpected va
By understanding these aspects, you can effectively use this code to determine the overall
'''

➞ overall physique of the team is: explosive

#question 2(g)

#If a new player is to be added into the team then find the maximum age allowed so that the

```
def max_age(a):
    ag=0
    length=len(a)
    for name,i in a.items():
        ag=ag+i['age']
    avag=ag/length
    max_allowed_age = (24 * (length + 1)) - avag
    max_allowed_age = (24 * (length + 1)) - ag
    return max_allowed_age

print(max_age(team))

...
```

Understanding the Code and the Problem

Problem:

The problem is to find the maximum age allowed for a new player to join a team such that :

Code Breakdown:

Function Definition:

`def max_age(a):` defines a function named `max_age` that takes a dictionary `a` as input, performs
Initialization:

`ag = 0`: Initializes a variable `ag` to store the total age of the team members.

`length = len(a)`: Calculates the number of team members.

Calculating Total Age:

`for name, i in a.items():`: Iterates through the dictionary `a`, where `name` is the player's name and `i` is the player's data.

`ag = ag + i['age']`: Adds the current player's age to the total age `ag`.

Calculating Average Age:

`avag = ag / length`: Calculates the average age of the team members.

Calculating Maximum Allowed Age:

`max_allowed_age = (24 * (length + 1)) - avag`: Calculates the maximum allowed age using a formula.

The second line `max_allowed_age = (24 * (length + 1)) - ag` overwrites the previous calculation.

Returning Maximum Allowed Age:

`return max_allowed_age`: Returns the calculated maximum allowed age.

Function Call:

`print(max_age(team))`: Calls the function `max_age` with the `team` dictionary and prints the result.

Issues and Potential Improvements:

The formula used to calculate `max_allowed_age` is unclear and seems incorrect.

The second calculation of `max_allowed_age` overwrites the first, which is likely unintended.

The code lacks comments to explain the purpose of the calculations.

Possible Interpretations and Improvements:

Without more context, it's difficult to determine the exact condition for calculating the

Maintaining an average age below a certain threshold. In this case, the formula might need a way to filter out players based on their positions. For example, if a coach wants to maintain a certain average age for players in a specific position, the formula would need to be adjusted to only consider players in that position.